مهمة عمل هندسية: TASK-0003-A (تحدي التصميم البرمجي)

وضع التحدي: بدون كود جاهز 🚀

المشروع: Social-Commerce-Hub | المستوى: Junior .NET Developer (Hands-on)

1. الهدف الهندسي والمطلوب منك برمجته

مطلوب منك كتابة كلاسين كاملين في مسار المشروع: `products/social-commerce-hub/src/backend/SocialCommerce.Domain/Entities/`:

1. **`Merchant.cs`**: يمثل التاجر وحسابه ومتجره على المنصة.
2. **`Product.cs`**: يمثل المنتج المعروض في المتجر وفي الفيد التفاعلي.

2. قواعد البيزنس الصارمة المطلوبة لكلاس التاجر (Merchant Invariants)

⚠ متطلبات التصميم:

- **البيانات الأساسية:** `Id` (Guid), `StoreName`, `Slug` (رابط المتجر), `Email`, `Balance` (الرصيد), `IsActive` (حالة المتجر), `CreatedAtUtc`.
- **شروط الإنشاء (Constructor Guard Clauses):**
  - اسم المتجر لا يمكن أن يكون فارغاً أو مسافات فقط.
  - الرابط (Slug) لا يمكن أن يكون فارغاً، ويجب تحويله دائماً لحروف صغيرة (Lowercase).
  - البريد الإلكتروني لا يمكن أن يكون فارغاً.
  - الرصيد المبدئي لأي تاجر جديد يجب أن يبدأ دائماً بصفر (0).
  - حالة المتجر تبدأ نشطة (`IsActive = true`).
- **سلوكيات العمل (Business Methods):**
  - دالة **إضافة رصيد (CreditBalance):** تقبل قيمة موجبة فقط؛ لو القيمة صفر أو سالبة ترفض وترمي استثناء.

- دالة **سحب/تسوية رصيد (PayoutBalance):** تخصم من رصيد التاجر، ولا تسمح بالسحب إذا كان المبلغ المطلوب أكبر من الرصيد الحالي للتاجر (تمنع الرصيد السالب).
- دالة **تجميد/إلغاء تنشيط المتجر (Deactivate):** تقوم بتعطيل المتجر.

3. قواعد البيزنس الصارمة المطلوبة لكلاس المنتج (Product Invariants)

⚠ متطلبات التصميم:

- **البيانات الأساسية:** `Id`, `MerchantId` (Guid), `Title`, `Price` (decimal), `MediaUrl`, `IsVideo` (bool), `StockQuantity` (int), `CreatedAtUtc`.
- **شروط الإنشاء:**
  - يجب التأكد من أن `MerchantId` ليس فارغاً (`Guid.Empty`).
  - عنوان المنتج لا يمكن أن يكون فارغاً.
  - السعر لا يمكن أن يكون سالباً (يسمح بصفر لو المنتج هدية/مجاني، ولكن ليس سالب).
  - رابط الميديا لا يمكن أن يكون فارغاً.
  - المخزون المبدئي لا يمكن أن يكون سالباً.
- **سلوكيات العمل (Business Methods):**
  - دالة **تعديل السعر (UpdatePrice):** تمنع وضع سعر سالب.
  - دالة **خصم المخزون عند الشراء (ReduceStock):** تقبل كمية مشتراة موجبة، وترفض العملية وترمي `InvalidOperationException` لو كانت الكمية المطلوبة أكبر من المخزون المتوفر.
  - دالة **إعادة تعبئة المخزون (Restock):** تضيف كمية للمخزون وترفض الكميات السالبة أو الصفرية.

4. فخاخ برمجية يقع فيها الـ Junior Developers (احذر منها!)

- **الفخ الأول:** ترك الخصائص `public set`. (ممنوع تماماً! استخدم `private set`).

- **الفخ الثاني:** استخدام `DateTime.Now` بدلاً من `DateTime.UtcNow`. (احرص على استخدام التوقيت العالمي الموحد).
- **الفخ الثالث:** نسيان الـ Parameterless Constructor الخاص بالـ EF Core. (اعمل `private Merchant() { }` بدون معاملات عشان الداتابيز تعرف تنشئه لاحقاً).
- **الفخ الرابع:** استخدام أنواع بيانات عائمة (Float/Double) للمبالغ المالية. (المبالغ المالية دائماً `decimal`).

5. قائمة الفحص الذاتي للتسليم

- [ ] الكلاسات تم إنشاؤها داخل مشروع `SocialCommerce.Domain`.
- [ ] قمت بتطبيق كل الـ Invariants والـ Business Methods المذكورة أعلاه.
- [ ] الـ Solution بيعمل `dotnet build` بدون أي Errors.

كده التحدي حقيقي وواقعي بدون أي حلول جاهزة!

بمجرد ما تكتب الكود وتتأكد منه، أرسل في الشات: **.NET Junior : انا خلصت**

وسأقوم أنا ومراجع الكود والـ QA بفحص ومراجعة حلك بدقة تامة وتقديم مراجعة شاملة.